DSA STUDY BLUEPRINT SERIES

80. Identical Tree Subtree of another Tree

Validating Tree Identity and Subtree matching

Section 09 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Identical:** Check if value fields match and recursive subtrees match.
- **Subtree:** Iterate tree nodes, executing identity checks for match candidates.

Memory & Execution Blueprint

C++ Implementation Reference

Standard code layout and syntax

```cpp
bool isSameTree(TreeNode* p, TreeNode* q) {
    if (!p && !q) return true;
    if (!p || !q) return false;
    return (p->val == q->val) && isSameTree(p->left, q->left) && isSameTree(p->right, q->right);
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Same Tree Check	O(N)
Subtree Matching	O(N * M) worst

Key Practices & Gotchas

- **Important:** Check null nodes correctly to avoid dereferencing segmentation faults.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace recursive Preorder Traversal visits (Node-Left-Right) for tree `1 (left=2, right=3)`:

Call Stack Level	Active Node Visited	Expected Left Traversal	Expected Right Traversal
1	Root (1)	Recurse on Left child (2)	Recurse on Right child (3)
2	Leaf (2)	Base case (nullptr)	Base case (nullptr)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Iterative traversals:** Standard DFS traversals can run iteratively by simulating recursion calls with Stacks.
- **Level Order BFS:** Level tracking searches use Queues to visit node layers.
- **Related LeetCode Problems:** #102 Binary Tree Level Order Traversal, #94 Binary Tree Inorder Traversal, #236 Lowest Common Ancestor.